# MASooda

## Final Project Report

### Multi-Agent System for Hierarchical, Priority-Aware, and Resilient Urban Air Traffic Management — as built, as demonstrated, and as measured

---

## 1. Scenario (as built)

A fleet of up to **500 heterogeneous autonomous drones** — emergency/medical drones (priority 0), passenger eVTOLs (priority 1) and parcel-delivery drones (priority 2) — shares low-altitude airspace over the **IIT Hyderabad campus (Kandi)** and **Hyderabad city roads**. Each drone is a self-interested agent that hears only neighbours within **40 m** over vehicle-to-vehicle (V2V) links and senses non-cooperative drones within 30 m. The system runs live at masooda.onrender.com.

The proposal imagined two or three straight corridors. The final airspace follows the **real road network**, because that is where drone corridors would actually be laid:

- **IITH campus map:** 28 roads traced from the campus map (~0.95 m/px) form a graph of 105 junctions with 13 vertiports — V1 Hospital, three depots (Mess A, Mess B, Sampoorna) and nine hubs (Main Gate, Guest House, Convention Centre, LHC, Admin Block, Academic Area, New Hostels, Old Hostels, SNCC) — enclosed by a geofence.
- **Hyderabad map:** OpenStreetMap major roads (motorway to tertiary) snapped into a routable graph of 1,533 junctions with 8 vertiports chosen by farthest-point sampling (hub, hospital, depot roles).
- **Corridor tubes:** each road is a 20 m tube with one keep-left lane per direction; a drone that drifts more than 10 m off the centreline is projected back onto the tube wall, and the map edge acts as a geofence.
- **2.5D layered airspace:** seven altitude layers L0–L6 stacked 15 m apart from 30 m to 120 m above ground (inside India's 400 ft green-zone ceiling). Following the semicircular rule, eastbound traffic uses even layers (L0, L2, L4), westbound odd layers (L1, L3, L5), and **L6 is reserved for medical emergencies**.
- **Sectors:** the airspace is split into six sectors (3 × 2 grid); every (sector, layer) cell has a nominal capacity of 15 drones.

The system must stay correct when agents behave **adversarially** (GPS-spoofed position broadcasts, a drone with no Remote ID), when the **controller link drops**, and when the physical world intervenes: **battery faults** (capacity loss, cell failure, overheating, motor and GNSS degradation), **heat** (thinner air at 35–44 °C raises hover power), **wind**, moving **storm cells** and pop-up **restricted areas**.

---

## 2. Objective — and What Was Achieved

All five design objectives of the proposal are implemented and run together, every tick ($\Delta t$ = 0.1 s), in one simulation loop:

1. **Local governance:** each drone follows its route by pure pursuit (look-ahead 40 m), keeps a car-following gap $v \le (g - d_0)/h$ with $d_0$ = 12 m and $h$ = 1 s, applies boids separation and alignment, and avoids neighbours by closest point of approach. Conflicting groups (same layer, within 20 m) negotiate their next positions with **consensus ADMM**; a crowded drone can also make a tactical altitude shift.
2. **Global governance:** the **Drone Traffic Controller (DTC)** runs every 2 s per sector: it counts (sector, layer) load, keeps emergencies on L6, moves drones out of over-capacity cells through a **safety gate** (a layer change is deferred if the target layer is occupied nearby), seeds layer congestion prices and executes operator commands. If the link fails, drones follow their lost-link policy (continue, hold, return to base or land) and operator commands wait in a queue for up to 30 s.
3. **Priority arbitration:** emergency > eVTOL > delivery is respected globally (reserved layer, priority-aware allocation) and locally (ADMM weights 10 / 3 / 1, and the lower-priority drone yields in a tactical shift). A drone whose battery management system (BMS) reports CRITICAL or FAULT is treated as priority 0 for right of way to land, and its landing pad is reserved.
4. **Adversarial resilience:** every received position report passes a **trusted-range check** (a report that disagrees with the sensed range by more than 3 m is moved along its bearing to that range) and **W-MSR** fusion with $f$ = 1; the operator can also quarantine a drone so its reports are ignored network-wide.
5. **Formal safety verification:** the STL specification $\varphi = \mathbf{G}(d_{ij}(t) \ge d_{min})$, $d_{min}$ = 8 m, is monitored live through its robustness $\rho(t) = \min_{i \ne j} d_{ij}(t) - d_{min}$, using a **layer-aware 3D distance** (horizontal offset plus layer gap × layer height), independent of which tier acted. A reachability spec $\mathbf{F}$(reach destination) is also implemented.

**Underlying research question:**

*Can a two-tier (decentralized-local / centralized-global) multi-agent architecture achieve scalable, priority-aware, and formally verifiable safe traffic management in dense UAM airspace, while remaining robust to adversarial or faulty agents — outperforming a purely centralized or purely decentralized alternative under the same conditions?*

**What our evaluation found (100 drones on the campus map, 20 paired seeds):**

*Hierarchical and decentralized control both delivered 95.7% of flights with zero near misses and a 12.5 m minimum separation; the centralized controller, acting on reports refreshed at its update rate, had 9.8 near misses and 1.1 collisions per run and delivered 88.0%. The hierarchy is as safe as decentralized control and clearly safer than central control. In nominal traffic its DTC did not add measurable safety on top of the local tier — its value shows in the scenarios it alone can handle: emergencies, ground stops, restricted areas, battery faults and operator commands.*

---

## 3. Outcomes Delivered

| # | Outcome (from the proposal) | How it is demonstrated now |
|---|---|---|
| 1 | A working simulation of N heterogeneous drone agents self-organizing into corridors under local rules, coordinated by a lightweight global overseer | Live map of the IITH campus or Hyderabad with drones coloured by type on road corridors, sectors and seven layers; a 2.5D side view per drone; DTC directives, layer changes and pad holds in the governance log. Fleets of up to 500 drones stream to the browser. |
| 2 | Demonstrated priority arbitration across drone classes | Preset 1 (medical emergency): two blood-delivery drones fly SNCC and Main Gate → Hospital on reserved layer L6 through a 120-drone lunch rush. Headless result: emergency compliance 100%, mean emergency delay 4.5 s, 0 near misses. The Safety panel shows compliance, delay against free flow and the yields imposed on other traffic. |
| 3 | Demonstrated resilience to adversarial agents | Preset 4 (GPS spoofing): two compromised drones broadcast positions 30 m off from 50 s; the map draws each spoofed report with an arrow to its W-MSR estimate; the operator quarantines both at 110 s. In the 20-seed evaluation the defence accepted 0 spoofed reports against 193 per run undefended. |
| 4 | Comparative evaluation: hierarchical vs. purely centralized vs. purely decentralized | Four architectures selectable in Setup and runnable from identical initial conditions (the IC hash is shown): hierarchical, centralized (DTC computes every velocity), decentralized (no DTC, ADMM + defence) and decentralized-undefended. Results in Section 5. |
| 5 | Formal safety guarantee | Live STL robustness plot ρ(t) in the Safety panel, plus near-miss episodes, violation ticks and crashes (< 2 m); separation alerts drawn on the map. |
| 6 | Quantified graceful degradation | Preset 5 (DTC outage 80–150 s) runs three lost-link policies side by side from the same seed. Headless result: continue 91 delivered · hold 76 delivered (+77% energy, emergency delay 51 s) · return-to-base 36 delivered. A speed limit sent at 125 s is queued and delivered when the link returns. |

---

## 4. How One Simulation Tick Works

Every module the team built runs inside the same loop (`simulation/route_engine.py`), in this order:

1. **Departures:** queued drones leave their pad when the pad airspace is clear and the corridor junction has a merge gap; otherwise they are held on the ground (logged, e.g. "ground hold at V10: pad airspace occupied").
2. **Neighbour search:** V2V graph within 40 m (spatial grid index, so 500 drones stay fast); non-cooperative drones are visible only to onboard sensing within 30 m.
3. **Attack and defence:** spoofers falsify their broadcasts; receivers apply the trusted-range check and W-MSR before using any report.
4. **Local behaviour:** route following, car-following, boids, closest-point-of-approach avoidance, junction merging and platoon tracking.
5. **ADMM:** conflict groups on the same layer within 20 m solve for their next positions; groups that do not converge in 50 iterations keep their intended velocities and count as fallbacks.
6. **Dynamics and boundaries:** velocity and acceleration limits (15 m/s), tube and geofence projection, crash detection (< 2 m).
7. **Battery and health (every 0.5–1 s):** power, pack and cell model, BMS estimate, feasibility checks, onboard responses.
8. **DTC (every 2 s) and layer agreement (every 1 s):** sector × layer allocation, emergency layer, pad arbitration, congestion prices, operator commands.
9. **Verification:** layer-aware STL robustness, near-miss episodes, KPIs; a compact frame streams to the browser.

---

## Mapping to Our Coursework (as implemented)

| Requirement | Course Modules | Where it lives in the code |
|---|---|---|
| Local rules / neighborhood behavior | Graph Theory Basics, Consensus & Topology Conditions | simulation/comm_graph.py (radius / kNN graph), local_behavior/boids.py; the Network panel reports degree, transitivity, components and algebraic connectivity $\lambda_2$ live |
| Centralized overseer + local tier interplay | Consensus Optimization, Distributed Learning & Tracking | conflict_resolution/ (consensus ADMM), simulation/overseer.py, simulation/layer_agreement.py (consensus-based dual ascent on layer prices) |
| Corridor merging | Formation Control, Vehicle Platooning Problem | local_behavior/, simulation/route_formation.py (junction merges, platoons, headway) |
| Priority arbitration | Epistemic Logic, Behavioural Planning with EL | simulation/overseer.py (priority-aware policy), simulation/route_priority.py (compliance, delay, yields) |
| Adversary handling | Malicious Attacks & Resilient Consensus | resilience/attack.py, defense.py, wmsr.py, consensus.py; W-MSR also trims layer prices |
| Formal safety guarantees | STL Basics, STL Robustness, Mission Planning with STL | verification/stl_monitor.py, live_monitor.py, metrics.py |
| Complex arbitration + safety together | Complex Mission Planning with STL & EL | Energy-feasibility test, pad arbitration and energy-margin ADMM weights in simulation/drone_health.py |

---

# Final Team Structure & Delivery

## Team Structure

Each person owned a **vertical slice** of one syllabus pillar. The last column lists what each person delivered, taken from **their own git commits** (106 commits, 15 Sep – 10 Nov 2026). P2's slice grew from the ADMM resolver into the simulator, demo and battery work that every other module now runs inside.

| # | Role | Primary Syllabus Topic(s) | Member | What was delivered (from git) |
|---|---|---|---|---|
| **P1** | Simulation Engine Lead | Graph Theory Basics + system integration | Dhadheechi + Abdur | **Dhadheechi:** Week 1 engine — agent dynamics with velocity / acceleration clamps, the shared AgentState schema, mock agents, uv project setup, top-down renderer and GIF animation. **Abdur:** the integrated simulation loop (engine.py), radius and kNN communication graph, environment and scenario, CSV logger, per-tick telemetry, the Week 4 MVP run and the Week 5 centralized architecture. |
| **P2** | Strategy & Conflict-Resolution Lead | Consensus Optimization, ADMM | Lalith | **ADMM:** consensus-ADMM conflict resolver — solver, constraints, convergence, adapter, conflict detection, N-agent groups (Weeks 1–4 files reached git through Sampad's integration commits), config keys, best-iterate tracking, E1 scaling experiment with a non-invasive profiler, and the Week 6 fixes that keep connected conflict groups whole and fall back cleanly on non-convergence. **2.5D airspace:** proposed the multi-layer airspace and built the first 7-layer ADMM demo (1 Nov), plus final_demo.py with SQLite run storage. **Road-corridor simulator:** route_engine.py (2,169 lines), road_network.py, spatial grid, corridor formation, priority and centralized modules for road maps; IITH campus map traced by hand; Hyderabad OSM map. **Live demo:** web_sandbox/ (streaming server, compact frames, browser UI, run export) deployed on Render; DTC operator console; 10 scenario presets; custom-drone builder. **Drone health:** power, battery, BMS estimator, faults, pad arbitration, energy-margin ADMM weights; layer agreement. **Docs and paper:** full project audit, integration plan, research-gap review, IEEE-style paper draft. |
| **P3** | Local Behavior & Formation Lead | Formation Control, Vehicle Platooning | Bhuvan | Boids formulation (PDF and an HTML prototype); real boids separation, alignment and cohesion on P1's graph; corridor following, target-corridor selection and merge-gap control; local-behaviour cache reset and the ADMM merge check; Week 5 formation metrics (merge success, throughput, corridor utilization) with a sweep script. |
| **P4** | Global Overseer & Priority Lead | Consensus & Topology, Epistemic Logic | Tanmay | Overseer contract and data model (Week 1); overseer skeleton with least-loaded and round-robin policies and tests (Week 2); priority-aware corridor allocation (Week 3); OverseerLink with outage windows and cached directives for graceful degradation (Week 4). |
| **P5** | Adversarial Resilience Lead | Malicious Attacks & Resilient Consensus | Sampad | Threat model (PDF); attack injection (constant and ramp offsets); plain-averaging consensus baseline; W-MSR (scalar and vector) and the report defence; W-MSR vs. baseline comparison; the decentralized-undefended architecture with experiments E4, E3 and E6; priority metrics; integration of 2.5D altitude layers into the grid engine; weekly merges of every role's branch. |
| **P6** | Formal Verification & Demo Lead | STL Basics, STL Robustness | Abdur + Dhadheechi | **Abdur:** live STL monitor wired into the simulation, near-collision check, standardized run metrics (22-column schema, recovery time, degradation score); Week 6 experiment harness, dashboard, plots and run_all scripts. The Week 1 STL monitor and toy trajectories were committed by Sampad. |

## Week-by-Week Roadmap (as executed)

| Week | P1 (Sim) | P2 (Strategy/ADMM) | P3 (Formation) | P4 (Overseer/Priority) | P5 (Resilience) | P6 (STL/Demo) |
|---|---|---|---|---|---|---|
| **1** | Dynamics, schema, renderer (Dhadheechi) | Two-agent ADMM QP solver + tests | Boids formulation, prototype | Overseer contract | Threat model, attack stub | STL specs, toy trajectories |
| **2** | Comm graph, engine loop (Abdur) | ADMM adapter, config, iteration sweep | Real boids on the graph | Overseer skeleton + tests | Plain consensus baseline | Robustness formulas |
| **3** | Live sim integration | N-agent ADMM, conflict detection | Corridor selection, merge gaps | Priority-aware allocation | W-MSR vs. baseline | Live monitor on sim runs |
| **4** | MVP run | Profiling; 7-layer 2.5D ADMM demo; final_demo.py; SQLite outputs | Cache reset, ADMM merge check | Overseer link resilience | Falsified broadcasts | STL events, near-collision |
| **4.5** | 2.5D altitude layers: idea and first 7-layer ADMM demo by Lalith (1 Nov); integrated into the grid engine with layer-aware ADMM and overseer by Sampad (4 Nov) | | | | | |
| **5** | Centralized baseline | E1 ADMM scaling + profiler | E2 formation metrics | E3 / E6 (committed by Sampad) | E4, undefended baseline | Standard metrics |
| **6** | Telemetry, harness | Audit of the repo; ADMM fixes (whole groups, clean fallback) | E2 plots | E3 / E6 plots | E4 plots | Dashboard, plots, run_all |
| **7** | Dynamics reused on road corridors | Road-corridor engine, IITH + Hyderabad maps, live demo phases 0–5, Render deployment | Boids + car-following on road tubes | Overseer per sector via OverseerLink | Range check + W-MSR on V2V reports | Layer-aware STL ρ(t) in the demo |
| **8** | Drone health and BMS, pad arbitration, energy-margin ADMM weights, layer agreement, multi-visitor server, light theme and section navigation, 460-run paper evaluation (P2) | | | | | |

## 1) Individual Contributions — What Each Person Did

Each statement is backed by that person's commits (git identities: `rlv-iith` and `R Lalith Vishnu` = Lalith; `MohammedAbdurRehman11015` = Abdur; `Sampadram` = Sampad).

- **P1 — Dhadheechi:** "I built the Week 1 simulation engine: agent dynamics, the shared schema every module uses, mock data, the project setup and the top-down renderer."
- **P1 + P6 — Abdur:** "I built the integrated simulation loop, the communication graph, telemetry and the centralized baseline, wired the STL monitor into live runs, defined the standard run metrics, and committed the Week 6 experiment harness, dashboard and plots."
- **P2 — Lalith:** "I designed and implemented the consensus-ADMM conflict resolver and measured how it scales. I proposed the 2.5D layered airspace, traced the IITH campus into a corridor graph, and built the road-corridor simulator that runs every team module. On it I built the live demo with its DTC console and scenario presets, the battery and BMS model with pad arbitration, and the layer agreement, and I wrote the project audit and the paper draft."
- **P3 — Bhuvan:** "I implemented boids alignment, cohesion and separation, corridor selection and merge-gap control, the ADMM merge check and the E2 formation metrics."
- **P4 — Tanmay:** "I built the overseer contract and skeleton, the priority-aware corridor allocation, and the overseer link model that lets the swarm keep flying on cached directives."
- **P5 — Sampad:** "I designed the threat model and implemented attack injection, the consensus baseline, W-MSR and the report defence, and the undefended baseline with experiments E4, E3 and E6. I integrated the 2.5D layers into the grid engine and merged every role's weekly branch."

---

## 2) What We Promised vs. What We Delivered

**Committed publicly (our "promise") — all delivered:**

- 2D (or 2.5D dual-panel) simulation → *2D map + 2.5D side view, seven altitude layers*
- 50–80 agents, 2–3 corridors → *up to 500 drones on full road networks (105-junction campus, 1,533-junction city)*
- One local rule set + one global overseer with priority arbitration → *boids, car-following and CPA avoidance; a sector-based DTC with a reserved emergency layer*
- One resilient consensus defense against one attack type → *trusted-range check + W-MSR against position spoofing, plus operator quarantine*
- One STL spec monitored live + one comparison chart → *live layer-aware ρ(t); four architectures from identical initial conditions*

**Headroom and stretch items:**

- Second STL spec (reachability) → *implemented* · corridor congestion pricing → *implemented (layer agreement with consensus prices)* · richer priority-conflict interface → *implemented (DTC console)*
- 500+ agents → *reached: 500 drones at 3.4× real time on a 2-vCPU server* · multiple attack types → *partly: spoofing and a non-cooperative intruder*
- True 3D rendering, full epistemic-logic belief modelling, learned (RL) policies → not done, as the proposal allowed

**Grown beyond the proposal during weeks 6–8:**

- Drones carry a physical **battery and BMS**, and the traffic manager uses the BMS state (priority to land, pad arbitration, energy-weighted ADMM).
- Weather (temperature, air density, wind), moving storm cells, temporary flight restrictions and ground-delay programs.
- A deployed web demo with an **operator console**, ten scripted scenarios and run export / replay; an IEEE-style paper draft with 460 paired-seed runs.

---

## 3) The Demo — What the Room Actually Sees

The final demo is **MASooda Live Airspace**: `python -m web_sandbox.server` (port 8765), deployed at `masooda.onrender.com`. The browser starts a run (`POST /api/runs`) and frames stream back while the simulator computes them. Every button below was exercised on the live site for this report.

### Top bar and playback

- **Status strip:** System ONLINE · Run LIVE · Sim time · Airborne / Queued / Landed / Forced / Crashed counters · DTC link UP/DOWN.
- **Playback:** pause (space), 1× / 2× / 4× / 8×, replay from start, a time scrubber under the map, **⬇ Run** (downloads the run as JSON: manifest, config, command log, KPIs, events, frames — built in the browser) and **⭱ Load** (replays a saved run).

### Right: the live airspace map

- Map switch **IITH campus (Kandi) / Hyderabad roads (OSM)** — switching launches a new run on that network; layer filter **All, L0–L6**; toggles for all routes, V2V links, sectors and the base map; zoom, fit and follow-selected-drone.
- Legend: emergency / eVTOL / delivery, corridor tube, selected route, vertiport, separation alert, crash, **spoofed report → W-MSR estimate**, junction merge (amber = gap blocked), BMS warning, BMS critical / fault, forced landing. Vertiports show their queue ("2 queued").

### Left: nine panel sections

| Section | What it shows and what each control does |
|---|---|
| Overview | One tile per section: run progress bar, safety (collisions, near misses, min separation, violation ticks), DTC (link, directive age, layer changes · gated, operator commands), fleet health (NORMAL / WARNING / CRITICAL / FAULT, lowest SoC, max pack temperature, diversions · forced, energy per delivery), traffic & priority (arrivals, emergency compliance and delay, junction merges, altitude shifts), ADMM, network, selected drone and custom drones. Clicking a tile opens its section. |

| | |
|---|---|
| Setup | Scenario preset (10, each with its own timeline and "watch" list) · architecture (hierarchical, centralized, decentralized, decentralized-undefended) · six quick variants (Normal ops, Emergency surge, DTC outage, Spoofing attack, Undefended attack, Decentralized) · fleet size (max 500), duration, departure window, seed · weather presets Winter 15 °C / Pre-monsoon 35 °C / Heat wave 42 °C with temperature, wind speed and direction and the resulting air density (1.074 kg/m³ at 35 °C → hover power +3.4%) · type mix (default 10% EMG, 30% eVTOL, 60% DEL) · module switches: P4 DTC, P2 ADMM, P3 junction merging + platoons, P5 W-MSR defence, P5 spoofing attack, P4 DTC link outage · model parameters from configs/base_config.yaml (cruise 12 m/s, $d_{min}$ 8 m, max 15 m/s, tube 20 m, capacity 15, V2V 40 m, ADMM radius 20 m, DTC period 2 s, Δt 0.1 s) · ▶ Launch. |
| Custom | Scenario builder: type and priority, origin and destination (13 vertiports or picked on the map), departure time, cruise speed, preferred layer (auto by semicircular rule, or L0–L6 with L6 for emergencies only), cooperative (broadcasts Remote ID) or malicious spoofer with an offset (dx, dy), battery Wh, initial SoC, state of health, payload, and a scheduled fault (capacity loss, cell failure, overheating, motor degradation, GNSS degradation) at a set time. **Add to scenario**, **Add drone now** (injects into the running simulation), import, and export as JSON or CSV. |
| DTC | Operator console. Select drones by click, Shift+click, Ctrl+drag box, right-click menu, or filters (type, sector, layer, airborne / queued) and drone-number ranges ("3, 7-12, 20"); keyboard H hold, R resume, G ground, B return, Q quarantine, [ ] layer. Commands: hold, resume, ground now, return to base, quarantine, make EMERGENCY / normal priority, change layer, reroute (with via-point), speed limit for drones or a sector, battery swap / charge at a pad, divert, lost-link policy per drone or fleet default; draw a restricted area with a time window; sector ground stop with release time; ground all BMS CRITICAL; recall below x% SoC. Every command asks for confirmation, then the command history shows received / delivered / acknowledged per drone. Below: the live sector × layer load table (6 × 7 cells, capacity 15, L6 marked as the emergency layer). |
| Drone | Pick any drone. **Overview** tab: mission and priority, route, layer and altitude, speed and heading, remaining distance and ETA, nearest same-layer drone, V2V degree, DTC link and directive age, ADMM negotiations, shifts and alerts, neighbour-report status. **Health** tab: SoC and BMS status, voltage · current · power (e.g. 22.3 V · 54.8 A · 1,219 W), energy left · time · range (259 Wh · 12.8 min · 5.2 km), state of health · capacity, pack and motor temperature against their limits, weakest cell and imbalance (3.698 V · 25 mV), DTC link RSSI and loss, GNSS fix and HDOP, motor health, faults; per-cell voltage bars and an SoC / temperature trace. Per-drone event log. |
| Health | Fleet BMS counts, the lowest-charge airborne drones (click to select), active alerts (e.g. "stuck at 4.4 m/s for 4 s on L0 — asks the DTC for a layer"), and the **battery matrix** of every drone: sort by id, lowest SoC or worst BMS; filter airborne / ≥ 30% / below 30% / below 15% or critical; each tile carries a 2-minute SoC sparkline. |
| Safety | Fourteen KPI tiles: crashes (< 2 m), near-miss episodes, 3D layer-aware minimum separation, violation ticks, ADMM resolutions (p95 iterations, ms, fallbacks), emergency compliance, junction merges (attempts, gap-yield ticks, platoons, headway), altitude shifts, DTC directives (layer changes, gated), corridor boundary holds, spoofed reports filtered, arrivals, energy per delivery, battery responses (diversions, forced, depleted, grounded pre-flight). The live STL robustness plot ρ(t). The governance log of every DTC, operator and BMS decision. For presets with variants, a **variant comparison** table of 14 KPIs side by side. |
| Network | The DTC hub plus the V2V graph: nodes and edges, average degree, density, transitivity, maximum degree centrality, components and isolated nodes, giant component share, algebraic connectivity $\lambda_2$; DTC link, directive age, directives issued, layer changes; a radar of coverage measures. |
| ADMM | P2 convergence timer: conflict groups, converged vs. fallback, iterations mean · p95 · max (of 50), solve time mean · p95, the current frame's conflict, and a chart of iterations and solve time over the run. |

## The ten scenario presets

| # | Scenario | What happens | Headless result (seed 42) |
|---|---|---|---|
| **1** | IITH medical emergency | Lunch-hour wave of 120 drones; @blood1 (SNCC) and @blood2 (Main Gate) fly to the Hospital on the reserved emergency layer with DTC right of way and ADMM against delivery traffic (240 s) | Compliance 100%, emergency delay 4.5 s, 0 near misses |
| **2** | Peak-hour vertiport congestion | 200 drones in a 60 s window; ground stops on S3 (to 90 s) and S4 (to 120 s); 8 m/s speed limit in S1 (300 s) | 223 ground holds, 161 / 200 delivered, 0 near misses |
| **3** | Pop-up restricted area (VIP) | TFR over Academic Area + LHC announced at 60 s, active 70–180 s; drones reroute or land; V5 / V7 departures held | 17 restricted-area events, 0 near misses |
| **4** | GPS spoofing near the restricted zone | Two drones broadcast positions 30 m off from 50 s; range check + W-MSR correct them; operator quarantines both at 110 s | 5 reports corrected, both quarantined, 1 near miss during the attack |
| **5** | DTC outage: lost-link procedures | Link down 80–150 s; variants continue / hold / rtb compared side by side; a command at 125 s is queued | Delivered 91 / 76 / 36; hold costs +77% energy |
| **6** | Weather cell | 250 m storm drifts east at ~10 m/s from the Main Gate with an 8 m/s westerly wind (300 s) | 120 restricted-area events, 3 caught inside landed, 0 near misses |
| **7** | Non-cooperative intruder | No Remote ID: invisible to DTC and ADMM; others sense it within 30 m; 6 m/s limit in S1 | Detected by onboard sensing, 0 crashes |
| **8** | Battery failure mid-route | @cargo1 loses a cell at 70 s; @cargo2 (31% SoC) loses 30% capacity at 80 s; operator grounds all CRITICAL at 85 s | @cargo1 right of way → diverted and landed; @cargo2 low-reserve diversion; 0 depleted |
| **9** | Heat wave 44 °C vs. 30 °C | Same 150-drone afternoon twice; two overheating faults at 60 s | Energy / delivery +4%; max motor 69.8 vs. 56.2 °C |
| **10** | Long-duration operations | 60 min, repeated sorties; depot battery swaps below 35% (2 per pad); 60 s outage at 20 min; recall below 25% at 40 min | 1,469 deliveries, 137 swaps, queue ≤ 5, 0 forced, 0 depleted |

**Observed live on the site (preset 8, battery failure), governance log in order:**

> BMS NORMAL → CRITICAL (CELL_IMBALANCE) · SoC 77% · pack 36 °C · motor 50 °C
> D-100 BMS CRITICAL (CELL_IMBALANCE): DIVERTING to V4 (328 m, needs 22 of 362 Wh)
> DTC: D-100 BMS CRITICAL → right of way to land (priority 0, stays on L0, pad V4 reserved)
> D-101 low reserve (57 Wh left, 66 Wh needed to reach V9): DIVERTING to V4 (265 m, needs 20 of 57 Wh)
> D-24 ground hold at V4: pad reserved for D-100 (BMS CRITICAL) landing
> DTC c1 fleet ground bms critical → D-100: 1/1 acknowledged — landing at V4

In a normal-operations run of 60 drones on the campus, about 60 s in, the Safety panel showed 130 ADMM groups all converged (p95 11 iterations, ≈4 ms), 0 near misses, 16.1 m minimum 3D separation, 100% emergency compliance and 58 junction merges at 100%. An operator hold sent to 12 selected drones came back "12/12 acknowledged" (airborne drones held position; queued drones had their departure held), and the resume was acknowledged the same way.

**The demo script (proposal beats → presets):**

1. **Normal operation (60 s):** any preset's opening minute — drones leave the pads, spread over the layers; Overview tiles healthy.

2. **Malicious drone + controller link drop (90 s):** presets 4 and 5 — spoofed reports corrected on the map, quarantine; drones keep flying on their lost-link policy and the queued command arrives when the link returns.
3. **Emergency drone (60 s):** preset 1 — blood delivery takes L6 and right of way; compliance and delay in the Safety panel.
4. **Comparison (30 s):** the architecture switch and the variant table — hierarchical vs. centralized vs. decentralized(-undefended).
5. **Battery failure (60 s):** preset 8 — the log above, live on the map.

---

## 4) Engineering Audit and Corrections

Before the final simulator was built, the repository was audited line by line against the workflow (docs/COMPREHENSIVE_PROJECT_AUDIT.md). It logged **21 discrepancies (D01–D21)**, five rated critical, nine high. The main ones and how the final system resolved them:

| ID | Finding | Resolution in the final system |
|---|---|---|
| **D01, D12** | STL and metrics measured horizontal (XY) distance while ADMM ignored other layers; z held a layer index, not metres | One layer-aware 3D distance (layer gap × layer height) drives ADMM, the STL monitor and the KPIs |
| **D05** | Conflict groups were split into chunks of 5, dropping conflict edges at chunk borders | Connected conflict components are solved whole (P2 Week 6 fix, tested) |
| **D06** | ADMM only ran inside the merge zone x = 700–800 m | ADMM runs anywhere on the map (merge_zone_only off in route mode) |
| **D07** | A non-converged ADMM iterate could be used as a resolution | Non-converged groups keep their intended velocities and are counted as fallbacks |
| **D09, D10** | No terminal behaviour; "arrival" meant crossing x = 1,400 m in any lane | Drones fly to a real destination pad, land and are counted once; arrivals, deliveries and forced landings are separate |
| **D11** | Corridors overflowed capacity with no throttling | Departures wait for a merge gap and a clear pad; sector × layer capacity with DTC moves and congestion prices |
| **D18** | Saved results showed 0.41 m minimum distance — ADMM is mitigation, not a guarantee | Reported as mitigation; an independent STL monitor and crash detection measure the outcome |

---

## 5) Results

The evaluation ran twice. **Week 6** ran experiments E1–E6 on the earlier three-corridor grid engine (141 runs, seeds 42–44, marked provisional). The **final evaluation** ran 460 paired-seed runs on the road-corridor engine. The final evaluation is the stronger evidence: the grid engine measured separation horizontally across layers (audit D01), which inflates its violation counts.

### Week 6 experiments (grid engine)

| Exp. | Owner | Question | Result |
|---|---|---|---|
| **E1** | P2 | ADMM scaling with N | 100% converged at every N (limit 50); ≈10 iterations and ≈1 ms per solve; ADMM time per tick 0.77 ms (N = 30) → 1.79 ms (N = 75); a limit of 20 already converges 99.7%, a limit of 5 only 7% |
| **E2** | P3 | Merge success, throughput | Throughput 0.16 → 0.42 agents/s from N = 30 to 75; merge success near 0; corridors peak at 1.8× capacity at N = 75 |
| **E3** | P4 | Priority compliance | Compliance 1.0 at N = 30, near 0 from N = 45 (assigned corridor over 15 agents); the emergency agent saves 10–36 s over a normal agent; delay imposed on others within ±2 s |
| **E4** | P5 | Resilience under attack | No measurable attack effect in any architecture (95% CI includes 0); undefended violations rise 0 → 76 ticks under attack |
| **E5** | P6 | STL robustness | Violating fraction: hierarchical 0.46–0.99, centralized 0.03–0.10, decentralized 0 (inflated by horizontal separation) |
| **E6** | P4 | Graceful degradation | Link down 90–120 s: directive age rose to 31.9 s; violations and throughput unchanged (score 1.0) |

### Final evaluation (road-corridor engine, 100 drones, 20 paired seeds)

| Measure | Result |
|---|---|
| Architectures (nominal) | Hierarchical & decentralized: 95.7% arrived, 0 near misses, 12.5 m min separation. Centralized: 88.0% arrived, 9.8 near misses, 1.1 collisions, 1.7 m min separation |
| Spoofing attack | 0 spoofed reports accepted (defended) vs. 193 per run (undefended); separation unchanged in both |
| DTC outage 80–150 s | 35 directives lost; no change in outcomes |
| Scaling 50 → 200 drones | Mean ADMM iterations ≈ 8.7, every group converged, p95 solve ≈ 0.45 ms; real-time factor 22.2× → 5.7× on one core; arrivals 95.8% → 90.9% |
| BMS state of charge | Estimator RMSE 0.21–0.33%; maximum error below 1.3% in all fault conditions |
| Energy bound | Never over-estimated usable energy in nominal flight; after a sudden 30% capacity loss, over-estimated in 9% of samples for about 2 minutes |
| Low-charge fleet | The feasibility check held 6.4 more drones on the ground and avoided 3.5 in-flight diversions per run; deliveries unchanged (−0.2) |
| Pad arbitration, weights | No measurable gain in safety, deliveries or emergency delay in these scenarios; one forced landing (seed 13) traced to where arrivals hold short |

### Live-demo performance (2-vCPU Xeon, IITH map, seed 42)

| Fleet × duration | Ticks / s | × real time | Peak airborne | Frame size |
|---|---|---|---|---|
| 60 × 180 s | 189.5 | 19.0 | 46 | 1.97 KB |
| 150 × 180 s | 87.9 | 8.8 | 81 | 3.91 KB |

| | | | | |
|---|---|---|---|---|
| 250 × 300 s | 63.4 | 6.3 | 114 | 5.90 KB |
| 500 × 300 s | 34.4 | 3.4 | 177 | 11.08 KB |

At 500 drones the campus is limited by vertiport capacity (464 ground holds), not by compute.

---

## 6) Testing, Reproducibility & Limitations

- **Tests:** 421 test functions in 80 files — unit tests per module (ADMM solver, boids, overseer, attack, W-MSR, STL), route-engine tests that assert every airborne drone stays inside its tube and the geofence on every tick, grid-on vs. grid-off identity, energy balance to $10^{-9}$, estimator calibration, pad arbitration, DTC commands, presets, KPI definitions and streaming.
- **Reproducibility:** one config file (`configs/base_config.yaml`) for every parameter; a seed fixes a run exactly; an initial-conditions hash proves architectures start identically; `scripts/run_all.py` regenerates Week 6; 53 runs repeated on a second platform gave identical discrete outcomes.
- **Limitations:** the battery, power and thermal models are not calibrated on flight data; health telemetry is trusted (a drone falsely reporting CRITICAL would get right of way); ADMM is a mitigation layer, not a formal guarantee; the threshold thermal response is too slow at 44 °C; wind changes power but does not push drones; V2V message loss is not modelled; Week 6 results are provisional.
- **Next steps:** calibrate against public delivery-drone flight data; predictive thermal and energy STL specs such as $\mathbf{G}(T_p \le T_{max})$; protect health reports the way W-MSR protects positions; scenarios with scarce landing capacity.

---

## 7) Repository Statistics (git)

106 commits across all branches (15 Sep — 10 Nov 2026). Lines are from `git blame` on 193 tracked code files (Python, JS, HTML, CSS outside `docs/` and `outputs/`), branch `bms/publishable` at `22c5400`.

| Member | Commits | Lines of code in the tree | Main areas |
|---|---|---|---|
| Lalith | 57 | 12,400 | simulation/route_*, road_network.py, drone_health.py, layer_agreement.py, dtc_console.py, web_sandbox/, ADMM profiler and fixes |
| Abdur | 11 | 12,023 (7,564 from the Week 6 commit) | simulation/engine.py, centralized.py, telemetry.py, verification/, visualization/, experiments/ |
| Sampad | 28 | 4,986 (≈1,660 in P2 files) | resilience/, integration merges, P2 Week 1–4 files, 2.5D layer integration |
| Bhuvan | 4 | 2,374 | local_behavior/, P3 scripts and tests |
| Dhadheechi | 1 | 1,102 | dynamics.py, schema.py, renderer.py, mock_data.py |
| Tanmay | 5 | 1,050 | overseer.py, overseer_link.py, P4 tests |

The final demo imports every role's module — P1 dynamics and comm graph, P2 ADMM, P3 boids, P4 overseer and link, P5 attack, defence and W-MSR, P6 STL metrics — so **every role's work runs in the final demo**.